

DevOps TP Finale

Contexte global

Vous rejoignez une startup qui développe un concept novateur de lacets connectés. Afin d'anticiper l'évolution rapide de la clientèle et pour permettre un suivi facile, une API a été développée. Vous êtes en charge de la mise en place des outils qui permettront de déployer automatiquement cette API.

Comme dans toute bonne startup, la documentation fournie avec le code source est minimaliste. Si vous avez des questions, n'hésitez pas à les venir me les poser. Les informations fournies sont volontairement limitée afin de vous encourager à creuser par vous même.

Sont attendus comme livrable :

- une documentation au format markdown dans un fichier README
- Une arborescence de fichier claire
- Tous les scripts fournis doivent fonctionner sur linux, pas de powershell ou AppleScript.

Il n'est pas attendu de fichier PDF, tout doit être mis dans le README. L'ensemble des éléments développés devront être le plus idempotent possible.

Pendant la dernière séance, vous ferez une démonstration de ce que vous avez développé. Ainsi l'évaluation fonctionnelle sera réalisée avec un environnement que vous maîtrisez complètement.

Partie 1 — Préparation de l'infrastructure (/20)

L'objectif de cette première partie est de mettre en place l'infrastructure qui servira ensuite à déployer l'application, pour se faire mettez en place les outils pour automatiser:

- Le déploiement d'une VM debian sur virtualbox (2Go de mémoire sont recommandées)
- Installation de k3s sur la VM

Si utilisation de DHCP :

- La récupération l'IP de la VM (Script bash (!\ pas de powershell ou bat))
- Création automatisée d'un inventaire ansible

Partie 2 — Conteneurisation de l'application (/10)

Récupérez le code source de l'application (<https://github.com/almoggutin/Node-Express-REST-API-MySQL-JS-Example>), créez une image docker puis poussez la sur docker hub.

Le soin apporté à l'optimisation de la taille de l'image sera pris en compte dans la notation.

Si besoin, vous êtes libre d'apporter les modifications que vous jugez nécessaire au code source. Prenez cependant soin de les documenter.

Partie 3 — Déploiement sur Kubernetes (/10)

Maintenant que vous avez une infrastructure fonctionnelle et une image docker, préparez les manifests kubernetes permettant le déploiement de l'API et d'une base de donnée. Prenez soin de faire en sorte que l'API soit joignable depuis l'intérieur du cluster (pas depuis l'extérieur) et que les données de la base de données soient persistantes.

Afin de garantir un fonctionnement fluide, il doit y avoir au moins 1 pod en permanence et jusqu'à 3 en cas de pic de charge (consommation de mémoire ou de CPU élevée)

Partie 4 — Mise en place du CI/CD (/20)

Maintenant que tout fonctionne, mettez en place une pipeline CICD sur github executant l'ensemble du processus de déploiement dès que la branche main est modifiée. Cette pipeline devra :

- Configurer l'infrastructure
- Builder l'image
- Déployer l'image buildée sur l'infrastructure
- Utiliser un runner self-hosted

Partie 5 — Monitoring et observabilité (/20)

Nous allons maintenant nous atteler au monitoring du serveur que nous venons de déployer. Pour se faire, modifier la pipeline CI/CD créée précédemment pour ajouter ceci :

- Une VM sur laquelle sont installée grafana et prometheus
- Installation de prometheus/node_exporter sur les deux VM (k3s et monitoring)
- Configuration de prometheus/node_exporter, prometheus et grafana pour que le dashboard ci-dessous soit accessible et qu'il affiche les données des deux serveurs.

<https://grafana.com/grafana/dashboards/1860-node-exporter-full/>